

HANNA Platform - Version 2.0.0 Release Notes

Release Date: February 1, 2026
Codename: Pfungwa
Project: HANNA - Installation Lifecycle Operating System

Executive Summary

Version 2.0.0 marks the official production release of **HANNA (Hanna Installation Lifecycle Operating System)** - a comprehensive, enterprise-grade CRM and lifecycle management platform purpose-built for Solar (SSI), Starlink (SLI), Custom Furniture (CFI), and Hybrid installations across Zimbabwe and the region.

Platform Capabilities:

-  **Multi-Portal Architecture:** 6 role-based portals (Admin, Client, Technician, Manufacturer, Retailer, Branch)
 -  **AI-Powered Automation:** Google Gemini AI for document processing and extraction
 -  **WhatsApp Integration:** Complete WhatsApp Business API with flow automation
 -  **E-Commerce Engine:** Full shopping cart, product catalog, and payment processing
 -  **Installation Management:** Complete ISR (Installation System Record) lifecycle tracking
 -  **Warranty System:** Multi-manufacturer warranty claims and service tracking
 -  **Enterprise Security:** HTTPS, Let's Encrypt SSL, HSTS, OCSP stapling
 -  **Real-time Monitoring:** WebSocket support for live updates and notifications
-

System Architecture

Technology Stack

Backend (Django 5.x):

- Django REST Framework - RESTful API layer
- Django Channels - WebSocket real-time communication
- Celery + Redis - Background task processing
- PostgreSQL - Primary database
- Daphne - ASGI server for WebSocket support
- Google Gemini AI - Document intelligence

Frontend Dashboards:

- **React 19 + Vite** - Admin dashboard (dashboard.hanna.co.zw)
- **Next.js 16.1.4 + TypeScript** - Management portals (hanna.co.zw)
- Tailwind CSS + shadcn/ui - Component library
- React Query - Server state management
- Zustand - Client state management

Infrastructure:

- Docker + docker-compose - Containerization
- Nginx - Reverse proxy and static file serving
- Certbot - Automated SSL certificate management
- Redis - Caching and message broker

Core Backend Apps (15+)

App	Purpose	Status
admin_api	Centralized admin API endpoints	✔ Complete
conversations	WhatsApp conversation management	✔ Complete
flows	WhatsApp flow automation engine	✔ Complete
meta_integration	WhatsApp Business API webhook integration	✔ Complete
products_and_services	Product catalog, cart, serialization	✔ Complete
customer_data	Customer profiles, orders, installations	✔ Complete
installation_systems	ISR lifecycle management	✔ Complete
warranty	Warranty claims and service tracking	✔ Complete
notifications	Unified notification system	✔ Complete
email_integration	IMAP/SMTP email automation	✔ Complete
ai_integration	Gemini AI document processing	✔ Complete
integrations	Zoho CRM sync	✔ Complete
paynow_integration	Payment gateway	✔ Complete
analytics	Business intelligence	✔ Complete
solar_integration	Solar monitoring	✔ Complete

 Major Features & Capabilities

1. Multi-Portal Architecture ✔

Six Role-Based Portals:

1. **Admin Portal** - Complete system governance
- Dashboard with KPIs and analytics
 - Full CRUD operations: Users, Products, Categories, Orders, Installations
 - Warranty claim management
 - Service request tracking
 - Installation pipeline (Kanban view)
 - Device monitoring and check-in/out
 - Flow management
 - System settings and notifications

2. **Client Portal** - Customer self-service

- Installation monitoring dashboard
- Warranty registration and claim submission
- Service request creation
- Order history and tracking
- Document access

3. **Technician Portal** - Field operations

- Assigned installations list
- Installation history
- Job card tracking
- Performance metrics
- Barcode scanning for check-in/out

4. **Manufacturer Portal** - Product and warranty management

- Warranty claim approval workflow
- Product tracking and analytics
- Barcode scanning
- Fault rate analytics

5. **Retailer Portal** - Sales distribution

- Solar package catalog
- Order management
- Installation tracking
- Branch management
- Commission tracking

6. **Branch Portal** - Local operations

- Inventory management
- Order dispatch
- Local customer service

2. AI-Powered Document Processing

Google Gemini AI Integration:

- **Automatic Invoice Processing** - Extracts structured data from invoice PDFs
- **Document Classification** - Identifies document type (invoice, job card, unknown)
- **Order Auto-Creation** - Creates orders with line items from invoices
- **Installation Request Generation** - Auto-creates linked installation requests
- **Customer Profile Management** - Auto-creates or links customer profiles
- **Job Card Processing** - Service document extraction
- **Smart Notifications** - Automated WhatsApp and email notifications
- **Duplicate Detection** - Prevents duplicate order creation

Email Pipeline:

Incoming Email → IMAP Fetch → PDF Extraction → Gemini AI Analysis → Document Classification → Data Structuring → Order/Installation Creation → Multi-Channel Notifications (WhatsApp + Email)

3. Installation System Record (ISR) Lifecycle

Complete Installation Management:

- **ISR Master Record** - Single source of truth for each installation
- **Multi-Type Support** - Solar, Starlink, Custom Furniture, Hybrid
- **Status Workflow** - pending → in_progress → commissioned → active → decommissioned
- **Digital Commissioning Checklist** - Type-specific step-by-step validation
- **Installation Photos** - Required photo evidence tracking
- **GPS Tracking** - Location coordinates for every installation
- **Component Tracking** - Links to serialized items and warranties
- **Technician Assignment** - Multi-technician support with scheduling
- **Automated Workflows** - Signal-based ISR creation from InstallationRequest
- **Installer Payout System** - Tiered payout rates with quality bonuses

Commissioning Features:

- Pre-install, installation, and commissioning phase checklists
- Required photo upload validation
- Progress tracking with completion percentages
- Hard validation: Cannot commission without 100% checklist completion
- Photo types: before, during, after, serial_number, test_result, site, equipment

4. Warranty & Service Management

Complete Warranty Lifecycle:

- **Serial Number Tracking** - Individual product tracking
- **Warranty Registration** - Automatic record creation during installation
- **Multi-Manufacturer Support** - Manufacturer-specific warranty claims
- **Claim Workflow** - Submission → Review → Approval → Resolution
- **Job Card System** - Service request tracking and assignment
- **Barcode Scanning** - Product check-in/out during service
- **SLA Tracking** - Service level agreement monitoring with alerts
- **Photo Documentation** - Fault evidence and resolution photos
- **Status Notifications** - Customer and technician notifications

5. WhatsApp Flow Automation

Comprehensive Flow Engine:

- **Flow Builder** - Visual flow editor with node-based interface
- **Dynamic Flows** - Variable interpolation and conditional routing
- **Customer Flows:**

- Solar installation request
- Starlink installation request
- Solar panel cleaning request
- Custom furniture delivery request
- Site assessment booking
- Loan application
- Product purchase (shopping cart)
- **Admin Flows:**
 - Order creation with installation
 - Order status updates
 - Assessment status updates
- **Human Handover** - Seamless bot-to-human transfer
- **Template Messages** - 24-hour window management
- **Interactive Messages** - Buttons, lists, forms

6. E-Commerce Platform

Digital Shop Features:

- **Product Catalog** - Complete product and category management
- **Shopping Cart** - Guest and authenticated cart support
- **Session Management** - Persistent carts across sessions
- **Stock Validation** - Real-time stock availability checking
- **Order Management** - Complete order lifecycle tracking
- **Payment Integration** - Paynow gateway integration
- **WhatsApp Shop** - Browse and purchase via WhatsApp flows
- **Price Management** - Dynamic pricing and promotions
- **Checkout Flow** - Streamlined purchase experience

7. Notification System (v2.0 Enhanced)

Unified Notification Infrastructure:

- **40+ Notification Templates** - Covering all lifecycle events
- **Multi-Channel Delivery** - WhatsApp, Email, SMS-ready
- **Template Categories:**
 - Order lifecycle notifications
 - Installation updates
 - Warranty and service alerts
 - Payment confirmations
 - System monitoring alerts
 - Admin operational notifications
 - Customer engagement messages
- **Meta API Integration** - WhatsApp Business API template sync
- **Variable Interpolation** - Dynamic content rendering
- **Group Notifications** - Role-based notification routing
- **24-Hour Window Management** - Template message compliance

Breaking Changes (v2.0)

1. Notification Template Rebranding REQUIRES DATABASE MIGRATION

All notification templates have been renamed from **hanna_*** prefix to **pfungwa_*** prefix:

Template Mapping:

Old Name (v1.x)	New Name (v2.0)
hanna_order_update	pfungwa_order_update
hanna_order_created	pfungwa_order_created
hanna_payment_successful	pfungwa_payment_successful
hanna_warranty_registered	pfungwa_warranty_registered
hanna_warranty_claim_update	pfungwa_warranty_claim_update
hanna_installation_scheduled	pfungwa_installation_scheduled
hanna_installation_completed	pfungwa_installation_completed
hanna_shop_order_confirmation	pfungwa_shop_order_confirmation
...and 32 more templates	Total: 40 templates renamed

Migration Required:

```
# Update NotificationTemplate model data in database
python manage.py shell
>>> from notifications.models import NotificationTemplate
>>> templates =
NotificationTemplate.objects.filter(template_name__startswith='hanna_')
>>> for t in templates:
...     t.template_name = t.template_name.replace('hanna_', 'pfungwa_')
...     t.save()
```

Code Changes:

-  All 26 backend files updated with new template references
-  All flow definitions updated
-  All signal handlers updated
-  All task processors updated
-  All test files updated

2. New Required Templates

Two new templates added to system (must be loaded into database):

1. **pfungwa_solar_alert_notification** - Solar monitoring alerts

- Parameters: `alert_title`, `alert_severity`, `alert_type`, `station_name`, `inverter_serial`, `description`, `occurred_at`
- Used by: `solar_integration/tasks.py`

2. `pfungwa_sla_alert` - Service SLA violation alerts

- Parameters: `request_type`, `request_id`, `customer_name`, `sla_deadline`, `current_status`, `days_overdue`
- Used by: `warranty/tasks.py`

Load New Templates:

```
python manage.py load_notification_templates
# Or
python manage.py seed_notification_templates
```

3. Meta WhatsApp Template Sync Required

All templates must be re-synced with Meta WhatsApp Business API:

```
# Sync templates to Meta API
python manage.py sync_whatsapp_templates
```

Important: Meta template approval can take 24-48 hours. Plan deployment accordingly.

New Features & Enhancements

Version Display in UI

-  React dashboard footer now displays "v2.0.0"
-  Next.js admin sidebar now displays "v2.0.0"
- Both frontends show "Pfungwa Technologies" branding

Enhanced Notification Templates (40 Total)

Customer Lifecycle Templates (15):

- `pfungwa_order_update` - Order status change notifications
- `pfungwa_order_created` - Order confirmation
- `pfungwa_payment_successful` - Payment receipt
- `pfungwa_warranty_registered` - Warranty activation
- `pfungwa_warranty_claim_update` - Claim status updates
- `pfungwa_installation_scheduled` - Installation appointments
- `pfungwa_installation_completed` - Installation confirmation
- `pfungwa_service_request_update` - Service tracking
- `pfungwa_delivery_scheduled` - Delivery appointments

- `pfungwa_shop_order_confirmation` - E-commerce purchases
- `pfungwa_welcome_message` - Customer onboarding
- `pfungwa_feedback_request` - Post-service feedback
- `pfungwa_appointment_reminder` - Scheduled appointments
- `pfungwa_assessment_completed` - Site assessment results
- `pfungwa_loan_application_update` - Loan status

Installation Lifecycle Templates (8):

- `pfungwa_installation_request_received` - Request acknowledgment
- `pfungwa_installation_request_approved` - Request approved
- `pfungwa_installation_request_rejected` - Request declined
- `pfungwa_installation_assessment_scheduled` - Assessment booking
- `pfungwa_installation_payment_pending` - Payment reminder
- `pfungwa_installation_in_progress` - Installation started
- `pfungwa_installation_rescheduled` - Date change notification
- `pfungwa_installation_cancelled` - Cancellation notice

System & Admin Templates (9):

- `pfungwa_admin_notification` - Internal admin alerts
- `pfungwa_task_assignment` - Technician assignments
- `pfungwa_technician_dispatch` - Job dispatch notifications
- `pfungwa_stock_alert` - Low stock warnings
- `pfungwa_payment_reminder` - Payment follow-ups
- `pfungwa_solar_alert_notification` - ☆ NEW: Solar monitoring alerts
- `pfungwa_sla_alert` - ☆ NEW: SLA violation alerts
- `pfungwa_system_maintenance` - Maintenance windows
- `pfungwa_broadcast_message` - Mass communication

Warranty & Service Templates (8):

- `pfungwa_warranty_claim_submitted` - Claim acknowledgment
- `pfungwa_warranty_claim_approved` - Claim approved
- `pfungwa_warranty_claim_rejected` - Claim declined
- `pfungwa_service_request_created` - Service ticket created
- `pfungwa_service_request_assigned` - Technician assigned
- `pfungwa_service_request_completed` - Service completed
- `pfungwa_job_card_created` - Job card issued
- `pfungwa_job_card_completed` - Job card closed

System Improvements

Flow Automation Enhancements:

- ☒ Improved error handling in flow processors
- ☒ Better logging for template rendering
- ☒ Enhanced variable validation
- ☒ Duplicate detection for flow responses

AI Document Processing:

- ☒ Enhanced Gemini AI prompts for better extraction accuracy
- ☒ Improved duplicate order detection
- ☒ Better error messages for failed processing
- ☒ Enhanced logging for debugging

Installation System:

- ☒ Enhanced ISR commissioning workflow
- ☒ Improved photo upload validation
- ☒ Better checklist progress tracking
- ☒ Enhanced installer payout calculations



Security Updates

Critical CVE Fixes

CVE-2025-55182: Next.js Remote Code Execution

- **Severity:** CRITICAL (CVSS 10.0)
- **Impact:** Server-side RCE vulnerability in Next.js < 16.1.4
- **Fix:** Upgraded to Next.js 16.1.4
- **Affected:** Management frontend (hanna-management-frontend/)

CVE-2025-66478: React Server Components RCE

- **Severity:** CRITICAL (CVSS 10.0)
- **Impact:** Remote code execution in React Server Components
- **Fix:** Upgraded to React 19.2.0 in Next.js app
- **Affected:** Management frontend

Additional Security Improvements

Dependency Upgrades:

- ☒ Axios 1.13.2 - Fixed DoS vulnerability
- ☒ React Router 7.12.0 - Fixed XSS vulnerabilities
- ☒ Vite 7.3.1 - Fixed file serving issues
- ☒ React 19.1.0 - Latest stable with security patches

Redis Hardening:

- ☒ Removed hardcoded Redis password
- ☒ Moved to environment variables
- ☒ Added `redis-password-reset.sh` script
- ☒ Updated docker-compose configuration

SSL/TLS Enhancements:

- ☒ Let's Encrypt SSL automation via Certbot

- ☒ TLS 1.2/1.3 enforcement
 - ☒ HSTS (Strict-Transport-Security) enabled
 - ☒ OCSP stapling configured
-

Performance & Infrastructure

Backend Optimizations

- ☒ Celery task queue optimization (separate IO and CPU queues)
- ☒ Database query optimization with select_related/prefetch_related
- ☒ Redis caching for frequently accessed data
- ☒ Static file compression via WhiteNoise
- ☒ Media file serving optimization via Nginx

Frontend Performance

- ☒ React 19 compiler optimizations
- ☒ Vite build optimization with code splitting
- ☒ Lazy loading for route components
- ☒ React Query caching and background refetching
- ☒ Image optimization and lazy loading

Database Improvements

- ☒ Indexed foreign keys across all models
 - ☒ Database constraints for data integrity
 - ☒ Optimized notification queue queries
 - ☒ Efficient ISR status tracking queries
-

Deployment Guide

Prerequisites

System Requirements:

- Ubuntu 20.04+ / Debian 11+ / RHEL 8+
- 4GB RAM minimum (8GB recommended)
- 50GB disk space
- Docker 20.10+ & docker-compose 2.x
- Domain with DNS configured

Environment Variables Required:

```
# Django
DJANGO_SECRET_KEY=<your-secret-key>
DEBUG=False
ALLOWED_HOSTS=yourdomain.com,www.yourdomain.com
CSRF_TRUSTED_ORIGINS=https://yourdomain.com,https://www.yourdomain.com
```

```
CORS_ALLOWED_ORIGINS=https://yourdomain.com

# Database
DB_NAME=hanna_db
DB_USER=hanna_user
DB_PASSWORD=<strong-password>
DB_HOST=db
DB_PORT=5432

# Redis
REDIS_PASSWORD=<strong-redis-password>

# Meta WhatsApp
WHATSAPP_TOKEN=<your-whatsapp-token>
WHATSAPP_PHONE_NUMBER_ID=<your-phone-number-id>
VERIFY_TOKEN=<your-webhook-verify-token>

# Google AI
GOOGLE_API_KEY=<your-gemini-api-key>

# Email
EMAIL_HOST=mail.yourdomain.com
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER=<email-user>
EMAIL_HOST_PASSWORD=<email-password>
IMAP_HOST=mail.yourdomain.com
IMAP_PORT=993

# Paynow
PAYNOW_INTEGRATION_ID=<paynow-integration-id>
PAYNOW_INTEGRATION_KEY=<paynow-integration-key>
```

Step-by-Step Deployment

1. Clone Repository & Configure Environment

```
git clone https://github.com/yourusername/hanna.git
cd hanna

# Copy and configure environment files
cp whatsappcrm_backend/.env.example whatsappcrm_backend/.env.prod
cp .env.example .env

# Edit environment files with your credentials
nano whatsappcrm_backend/.env.prod
nano .env
```

2. SSL Certificate Setup

```
# Run SSL setup script
chmod +x setup-ssl-certificates.sh
./setup-ssl-certificates.sh

# Follow prompts to configure your domains
# Certificates will be auto-renewed via Certbot
```

3. Build and Start Services

```
# Build Docker images
docker-compose build

# Start services in detached mode
docker-compose up -d

# Verify all services running
docker-compose ps
```

4. Database Migration & Setup

```
# Run database migrations
docker-compose exec backend python manage.py migrate

# Create superuser
docker-compose exec backend python manage.py createsuperuser

# Collect static files
docker-compose exec backend python manage.py collectstatic --noinput

# Load notification templates (CRITICAL FOR v2.0)
docker-compose exec backend python manage.py load_notification_templates

# Sync templates with Meta WhatsApp API
docker-compose exec backend python manage.py sync_whatsapp_templates
```

5. Verify Installation

```
# Check backend logs
docker-compose logs -f backend

# Check Celery workers
docker-compose logs -f celery_io_worker
docker-compose logs -f celery_cpu_worker

# Check Nginx
```

```
docker-compose logs -f nginx

# Test HTTPS access
curl -I https://yourdomain.com
```

6. Post-Deployment Verification

Backend Health:

```
# Django admin access
https://backend.yourdomain.com/admin/

# API health check
https://backend.yourdomain.com/api/health/

# WebSocket connection test
wss://backend.yourdomain.com/ws/notifications/
```

Frontend Access:

```
# React dashboard
https://dashboard.yourdomain.com

# Next.js management
https://yourdomain.com

# Verify version display shows "v2.0.0"
```

Database Verification:

```
docker-compose exec backend python manage.py shell

>>> from notifications.models import NotificationTemplate
>>> templates =
NotificationTemplate.objects.filter(template_name__startswith='pfungwa_')
>>> print(f"Found {templates.count()} pfungwa templates")
# Should show 40 templates

>>> # Check new templates exist
>>>
NotificationTemplate.objects.get(template_name='pfungwa_solar_alert_notification')
>>> NotificationTemplate.objects.get(template_name='pfungwa_sla_alert')
```

Upgrading from v1.x

Pre-Upgrade Backup

```
# Backup database
docker-compose exec db pg_dump -U hanna_user hanna_db > backup_v1.sql

# Backup media files
tar -czf mediafiles_backup.tar.gz whatsappcrm_backend/mediafiles/

# Backup environment files
cp whatsappcrm_backend/.env.prod .env.prod.backup
```

Upgrade Process

```
# Pull latest code
git pull origin main

# Rebuild containers
docker-compose down
docker-compose build
docker-compose up -d

# Run migrations
docker-compose exec backend python manage.py migrate

# CRITICAL: Rename notification templates in database
docker-compose exec backend python manage.py shell
>>> from notifications.models import NotificationTemplate
>>> templates =
NotificationTemplate.objects.filter(template_name__startswith='hanna_')
>>> for t in templates:
...     t.template_name = t.template_name.replace('hanna_', 'pfungwa_')
...     t.save()
>>> exit()

# Load new templates
docker-compose exec backend python manage.py load_notification_templates

# Re-sync with Meta API
docker-compose exec backend python manage.py sync_whatsapp_templates

# Restart services
docker-compose restart backend celery_io_worker celery_cpu_worker
```

Post-Upgrade Verification

```
# Verify template migration
docker-compose exec backend python manage.py shell
```

```
>>> from notifications.models import NotificationTemplate
>>> old_count =
NotificationTemplate.objects.filter(template_name__startswith='hanna_').count()
>>> new_count =
NotificationTemplate.objects.filter(template_name__startswith='pfungwa_').count()
>>> print(f"Old templates: {old_count}, New templates: {new_count}")
# Should show: Old templates: 0, New templates: 40

# Test notification sending
>>> from notifications.services import queue_notifications_to_users
>>> # Test with a sample notification
```

Testing & Quality Assurance

Backend Testing

```
# Run all tests
docker-compose exec backend python manage.py test

# Test specific apps
docker-compose exec backend python manage.py test notifications
docker-compose exec backend python manage.py test flows
docker-compose exec backend python manage.py test installation_systems

# Test notification templates
docker-compose exec backend python run_tests.py
```

Frontend Testing

```
# Dashboard (React)
cd whatsapp-crm-frontend
npm run lint
npm run build
npm run preview

# Management (Next.js)
cd hanna-management-frontend
npm run lint
npm run build
npm start
```

Integration Testing

```
# WhatsApp webhook test
curl -X POST https://backend.yourdomain.com/api/whatsapp/webhook/ \
```

```
-H "Content-Type: application/json" \  
-d '{"test": "data"}'  
  
# Notification queue test  
docker-compose exec backend python manage.py shell  
>>> from notifications.services import queue_notifications_to_users  
>>> # Test notification sending logic  
  
# AI document processing test  
docker-compose exec backend python test_media_urls_fix.py
```

Documentation

Core Documentation Files

Setup & Deployment:

- [README.md](#) - Main project documentation
- [DEPLOYMENT_INSTRUCTIONS.md](#) - Comprehensive deployment guide
- [SSL_SETUP_GUIDE.md](#) - SSL certificate configuration
- [SECURITY_IMPROVEMENTS.md](#) - Security best practices
- [CREDENTIALS_ROTATION_GUIDE.md](#) - Password rotation procedures

Feature Guides:

- [docs/features/ECOMMERCE_IMPLEMENTATION.md](#) - E-commerce platform
- [docs/features/GEMINI.md](#) - AI document processing
- [docs/features/NOTIFICATION_SYSTEM_README.md](#) - Notification infrastructure
- [docs/features/SHOP_NOW_README.md](#) - Shopping features
- [FLOW_INTEGRATION_GUIDE.md](#) - WhatsApp flow development

Architecture:

- [docs/architecture/ISR_IMPLEMENTATION_STATUS.md](#) - Installation system
- [docs/architecture/FLOW_DIAGRAMS.md](#) - System flow diagrams
- [docs/architecture/ISR_QUICK_REFERENCE.md](#) - ISR quick guide
- [SYSTEM_ALIGNMENT_REPORT.md](#) - Backend app analysis

Operations:

- [ADMIN_CRUD_QUICK_REFERENCE.md](#) - Admin operations guide
- [docs/troubleshooting/](#) - Common issues and solutions

API Documentation

Admin API Endpoints:

GET	/api/admin/dashboard/	- Dashboard metrics
GET	/api/admin/users/	- User management

GET	/api/admin/products/	- Product catalog
GET	/api/admin/orders/	- Order management
GET	/api/admin/installations/	- Installation pipeline
GET	/api/admin/warranties/	- Warranty claims
POST	/api/admin/notifications/send/	- Send notifications

Customer API Endpoints:

GET	/api/customer/profile/	- Customer profile
GET	/api/customer/orders/	- Order history
GET	/api/customer/installations/	- Installation status
POST	/api/customer/warranty/claim/	- Submit warranty claim
GET	/api/cart/	- Shopping cart
POST	/api/cart/add/	- Add to cart

WhatsApp API:

POST	/api/whatsapp/webhook/	- Meta webhook endpoint
POST	/api/whatsapp/send-message/	- Send WhatsApp message
POST	/api/whatsapp/send-template/	- Send template message
GET	/api/flows/	- List available flows



Known Issues & Limitations

Current Limitations

1. Meta Template Approval Timing

- Template sync requires 24-48 hours for Meta approval
- Plan deployments accordingly
- Workaround: Pre-approve templates in Meta Business Manager

2. Notification Template Migration

- Manual database update required for v1.x → v2.0 upgrade
- Automated migration script planned for v2.1
- Follow upgrade guide carefully to avoid notification failures

3. AI Document Processing

- Gemini AI accuracy depends on PDF quality
- Scanned documents may require OCR preprocessing
- Complex invoice layouts may need manual verification

4. WebSocket Connection Limits

- Daphne concurrent connection limit: 1000 per worker

- Scale horizontally for >1000 concurrent users
- Monitor `docker-compose logs -f backend` for connection warnings

Workarounds

Meta Template Sync Failures:

```
# If sync fails, manually verify in Meta Business Manager
# Then force re-sync:
docker-compose exec backend python manage.py shell
>>> from meta_integration.services import sync_templates
>>> sync_templates(force=True)
```

Redis Connection Issues:

```
# If Redis password changed, restart all services:
docker-compose restart
# Verify with:
docker-compose exec backend python manage.py shell
>>> import redis
>>> r = redis.Redis(host='redis', password='your-password')
>>> r.ping()
# Should return: True
```

HANNA Strategic Roadmap (v2.0 - v3.0+)

Roadmap Principles:

-  **Aligned with Core Scope** - All features directly support Installation Lifecycle Operating System vision
-  **Not Limiting** - Roadmap guides but allows for opportunities and market feedback
-  **Predictable Releases** - Monthly minor versions, quarterly major versions
-  **Iterative Refinement** - Each release builds on previous foundations
-  **Internal Focus** - Practical features for Pfunwa operations

Internal Roadmap (Post-v2.0)

HANNA Product Roadmap: v2.1 - v3.0

Q1 2026 - Stabilization & Core Operations

Q1 2026 - Strategic Transformation & Stabilization

v2.1.0 - January 2026 ( Strategic Shift: CRM → Installation Lifecycle OS)

Focus: Transformational release - evolution from pure CRM to comprehensive Installation Lifecycle Operating System

Strategic Shift:

- **✗ BEFORE:** HANNA as WhatsApp CRM with basic installation tracking
- **✓ AFTER:** HANNA as Installation Lifecycle Operating System (ISO) - complete installation lifecycle from sales through monitoring

Major Features (Core to New Vision):

- **✓ Installation System Record (ISR) - Full Implementation**
 - Master object for all installations with lifecycle tracking
 - Support for **4 installation types:** Solar (SSI), Starlink (SLI), Custom Furniture (CFI), Hybrid
 - Status workflow: pending → in_progress → commissioned → active → decommissioned
 - Links to customer, order, technicians, components, warranties, job cards
 - GPS coordinate tracking and digital installation file
 - Installation classification and system sizing
- **✓ Solar System Record (SSR) - Founding Framework**
 - Specialized ISR template for solar installations
 - Inverter integration framework (Growatt, Victron, Deye)
 - System size and capacity tracking
 - Energy generation monitoring hooks (API-ready)
 - Solar-specific commissioning requirements
- **✓ Digital Commissioning Checklists - Complete System**
 - Template-based checklists per installation type
 - Three phases: pre-install, installation, commissioning
 - Progress tracking with completion percentages
 - **Hard validation:** Cannot commission without 100% checklist completion
 - Required photo enforcement
 - Notes and observations per item
 - Technician sign-off and timestamps
- **✓ Installation Photo Documentation System**
 - 7+ photo types: before, during, after, serial_number, test_result, site, equipment
 - Type-specific requirements per installation type
 - Solar: serial_number, test_result, after (minimum)
 - Starlink: serial_number, equipment, after
 - Hybrid: serial_number, test_result, equipment, after
 - Furniture: before, after
 - Photo metadata and timestamps
 - Gallery view and organization
- **✓ Installer Payout System - Complete Framework**

- Tiered payout rates by installation type and system size
- Quality bonuses for exceptional work
- Payout calculation engine
- Payment approval workflow
- Configuration per installation type
- **✔ Notification Template Rebranding (Secondary)**
 - All 40+ templates renamed hanna_* → pfungwa_*
 - Unified template system across backend
 - Multi-channel support (WhatsApp, email)
 - 26 files updated for consistency

Data Model Foundation:

- **InstallationSystemRecord** - Master object (UUID, short_id ISR-xxxxxxx)
- **CommissioningChecklistTemplate** - Reusable templates
- **InstallationChecklistEntry** - Per-installation checklist instances
- **InstallationPhoto** - Photo evidence tracking
- **PayoutConfiguration** - Tiered payout rules
- Support for SSI, SLI, CFI, Hybrid types

System Readiness:

- **✔** Backend: 100% complete (all models, APIs, validation)
- **✔** APIs: Full CRUD endpoints for ISR, checklists, photos, payouts
- **⚠** Frontend: Basic list views (ISR list page exists), detail/edit pages in v2.2
- **✔** Notifications: 40 templates unified and operational
- **✔** Automation: Signal-based ISR creation from InstallationRequest

Impact: This release transforms HANNA from a CRM system to an **Installation Lifecycle Operating System** capable of managing the complete journey of installations from initial request through commissioning, warranty, monitoring, and service. It establishes the foundation for all future enhancements in solar monitoring, technician field operations, and predictive maintenance.

Version Display:

- **✔** React dashboard footer: "v2.1.0"
- **✔** Next.js admin sidebar: "v2.1.0"
- **✔** Branding: "Pfungwa Technologies"

v2.1.1 - February 2026 (Backend Stability)

Focus: Performance optimization, reliability improvements

Features:

- **✔ Database Optimization**
 - Query performance monitoring and logging
 - Slow query identification and optimization

- Index analysis for high-volume tables (conversations, orders, installations)
- Connection pooling setup (optional for scale)
- **✓ Task Queue Reliability**
 - Celery task retry strategy implementation (exponential backoff)
 - Dead letter queue for failed critical tasks
 - Task progress tracking API endpoint (/api/admin/tasks/<task_id>/status/)
 - Improved error logging and alerting
- **✓ API Stability**
 - Rate limiting per user (100 requests/minute standard users, 1000/minute admins)
 - Request timeout optimization (prevent hanging requests)
 - Graceful error handling for edge cases
 - Response time monitoring dashboard

Technical Debt:

- Add database indexes for foreign key lookups
- Optimize notification query performance
- Add caching for frequently accessed data (products, categories)

v2.1.2 - March 2026 (Frontend Polish)

Focus: User experience enhancements, accessibility

Features:

- **✓ UI/UX Improvements**
 - Faster page load times (target: <2s for admin pages)
 - Mobile responsiveness audit and fixes
 - Consistent component styling across portals
 - Improved error messages with actionable guidance
- **✓ Accessibility**
 - WCAG 2.1 Level A compliance audit
 - Color contrast improvements
 - Keyboard navigation enhancements
 - Screen reader testing (NVDA, JAWS)
- **✓ Documentation**
 - User guides for each portal
 - Video tutorials for key workflows
 - Troubleshooting documentation
 - API documentation auto-generation

Testing:

- Manual accessibility testing
 - Cross-browser testing (Chrome, Firefox, Safari, Edge)
 - Mobile device testing (iOS, Android)
-

Q2 2026 - Feature Expansion & Monitoring

v2.2.0 - April 2026 (Installation System Completion)

Focus: Complete ISR frontend implementation, technician field tools

Features:

- ☒ **ISR Frontend Completion** (Backend exists 100%)
 - ISR detail page with full record view
 - ISR edit page with status management
 - Commission installation workflow (requires 100% checklist + photos)
 - Installation photos gallery with type filtering
 - Commissioning checklist progress tracking
- ☒ **Technician Portal Enhancements**
 - Real-time checklist item completion with photo capture
 - GPS-based location tracking for field technicians
 - Offline-first checklist functionality (sync when online)
 - Photo upload with automatic compression
 - Digital signature capture for handoff
- ☒ **Installation Workflow Automation**
 - Automatic ISR creation from InstallationRequest
 - Status synchronization between frontend and backend
 - Phase transition validation (pre-install → install → commissioning)
 - Photo requirement validation per installation type

Data Model:

- ISR supports: Solar (SSI), Starlink (SLI), Custom Furniture (CFI), Hybrid
- Photo types: before, during, after, serial_number, test_result, site, equipment
- Checklist phases: pre_install, installation, commissioning

v2.2.1 - May 2026 (Solar System Monitoring)

Focus: Solar inverter integration and energy analytics

Features:

- ☒ **Solar System Tracking**
 - Basic inverter data display (generation, voltage, current, faults)
 - Real-time alert notifications for critical faults

- Daily/weekly/monthly energy generation dashboards
- Performance alerts (below-expected generation)
- Integration with Growatt and Victron inverters (APIs)
- **✓ Installation Monitoring Dashboard**
 - Real-time WebSocket updates for installation progress
 - GPS-based technician location mapping
 - Installation status notifications (started, completed, commissioned)
 - Technician performance analytics (completions per day, average time)
- **✓ Alert System**
 - Automatic alert generation for system faults
 - Alert routing (admin, customer, technician)
 - Alert acknowledgment workflow
 - Alert history and analytics

Technical Implementation:

- WebSocket endpoints for real-time updates
- Background tasks for periodic inverter data fetch
- Alert rules engine (configurable thresholds)

v2.2.2 - June 2026 (Analytics & Reporting)

Focus: Business intelligence, performance metrics, decision support

Features:

- **✓ Business Intelligence Dashboards**
 - Sales dashboard (orders by month, revenue, top products)
 - Installation dashboard (completion rates, average time, by technician)
 - Warranty dashboard (claims by product, approval rates, SLA adherence)
 - Customer analytics (retention, order frequency, satisfaction)
- **✓ Performance Reports**
 - Technician performance scorecard (installations completed, average commission time, photo requirement compliance)
 - Product performance analysis (defect rates, warranty claims, customer satisfaction)
 - Retailer/manufacturer performance metrics
 - Branch-level analytics
- **✓ Custom Report Builder**
 - Drag-and-drop report creation
 - Flexible date range selection
 - Export to PDF, CSV, Excel
 - Scheduled report generation and email delivery

Metrics to Track:

- Installation completion time (target: <2 weeks)
 - Photo compliance rate (target: 100%)
 - Checklist completion rate (target: 100% before commissioning)
 - Customer satisfaction (NPS score)
 - Warranty claim rate (by product)
-

Q3 2026 - Integration & Scalability**v2.3.0 - July 2026 (Warranty Management Excellence)**

Focus: Complete warranty lifecycle, claims processing automation

Features:

- ☒ **Warranty Claims Automation**
 - AI-powered claim review (suggest approval/rejection based on patterns)
 - Automated eligibility checking against warranty terms
 - Document upload requirements enforcement
 - SLA tracking and escalation alerts
- ☒ **Manufacturer Portal Enhancement**
 - Dashboard showing all claims requiring attention
 - Claim detail view with all documentation
 - Batch approval/rejection workflow
 - Claim trend analysis by product
- ☒ **Warranty Certificate Management**
 - Digital warranty certificate generation (PDF)
 - QR code linking to warranty record
 - Certificate validity tracking
 - Warranty transfer support (when product transferred to new owner)
- ☒ **Service Request Tracking**
 - Complete lifecycle: creation → assignment → in-progress → completion
 - Service history for each product/installation
 - Parts and labor tracking
 - Customer feedback collection post-service

v2.3.1 - August 2026 (Zoho CRM & Payment Integration)

Focus: Operational integration with Zoho, improved payment processing

Features:

- ☒ **Zoho CRM Sync**

- Real-time bidirectional sync (HANNA ↔ Zoho)
- Customer data sync (name, phone, email, address)
- Order sync (creation → Zoho deals)
- Warranty claim sync (to Zoho tickets)
- Two-way conflict resolution
- **✓ Payment Processing**
 - Paynow payment gateway integration (already exists, enhanced)
 - Multiple payment method support (credit card, bank transfer, mobile money)
 - Invoice generation and PDF export
 - Payment reminder system (automatic WhatsApp + email)
 - Payment tracking per order
- **✓ Email Integration Enhancements**
 - Improved IMAP/SMTP reliability
 - Invoice extraction from received emails (Gemini AI)
 - Automatic order creation from invoices
 - Email template management for notifications

v2.3.2 - September 2026 (Multi-Location Operations)

Focus: Branch management, inventory distribution, operational scalability

Features:

- **✓ Branch Management**
 - Branch-specific inventory tracking
 - Inter-branch stock transfers
 - Branch-level user permissions
 - Branch revenue and metrics
- **✓ Retailer Portal Enhancements**
 - Inventory management by location
 - Order dispatch to specific branches
 - Branch performance analytics
 - Stock reorder automation
- **✓ Operational Scalability**
 - Load balancing for backend services
 - Database read replicas for reporting
 - Cache optimization (Redis)
 - CDN setup for media files

Q4 2026 - Advanced Features & Intelligence

v2.4.0 - October 2026 (E-Commerce & Shopping Experience)

Focus: Complete e-commerce platform, shopping flows

Features:

- **✔ Shopping Cart Enhancements**
 - Persistent carts (guest + authenticated)
 - Wishlist functionality
 - Product recommendations (based on order history)
 - Quick reorder for returning customers
- **✔ Product Management**
 - Bulk product operations (import, pricing, category update)
 - Product variants and options
 - Product barcode management
 - Inventory sync with serialized items
- **✔ Checkout & Ordering**
 - Multi-step checkout process
 - Order tracking with notifications
 - Multiple delivery address support
 - Order history and analytics
- **✔ WhatsApp Shop Integration**
 - Browse and purchase via WhatsApp flows
 - Product images and descriptions in flows
 - Checkout confirmation via WhatsApp
 - Order status updates via WhatsApp

v2.4.1 - November 2026 (Client Portal Excellence)

Focus: Customer self-service, monitoring, warranty access

Features:

- **✔ Installation Monitoring for Customers**
 - Live installation progress tracking
 - Solar system performance dashboard
 - Energy generation insights and comparisons
 - Technician contact and scheduling
- **✔ Warranty & Service Self-Service**
 - Warranty registration self-service
 - Warranty claim submission interface
 - Service request creation

- Document upload for claims
- Claim status tracking
-  **Customer Notifications**
 - Installation phase updates
 - Warranty registration confirmation
 - Service request status updates
 - Energy performance alerts
 - Maintenance reminders

v2.4.2 - December 2026 (Admin Excellence & Year 2 Planning)

Focus: Admin portal completeness, year-end reporting, 2027 planning

Features:

-  **Admin Dashboard**
 - Company-wide KPI summary
 - Key metrics (installations, orders, revenue, warranty)
 - Alert dashboard (SLA violations, system alerts)
 - Quick action items
-  **Configuration Management**
 - System settings and configuration
 - User role and permission management
 - Email and SMS configuration
 - WhatsApp template management
 - Backup and recovery settings
-  **Year-End Reporting**
 - Annual performance report generation
 - Financial summary (revenue, costs, margins)
 - Customer acquisition and retention metrics
 - Employee/technician performance reviews

Q1 2027 - Next-Generation Features

v3.0.0 - January 2027 (AI-Powered Operations)

Focus: Intelligent automation, predictive capabilities, smart routing

Features:

-  **Predictive Maintenance**
 - ML model for equipment failure prediction
 - Proactive maintenance scheduling

- Spare parts recommendation
- Service cost estimation
- **✔ Smart Technician Routing**
 - AI-powered job assignment based on location, skills, capacity
 - Optimal route planning for multiple jobs per day
 - Travel time optimization
 - Technician workload balancing
- **✔ Customer Intelligence**
 - Churn prediction model
 - Lifetime value estimation
 - Personalized offers and recommendations
 - Service need prediction (when customer likely needs service)
- **✔ Document Processing Intelligence**
 - Enhanced invoice extraction (Gemini AI v2)
 - Multi-language support (Shona, Ndebele, English)
 - Handwriting recognition for job cards
 - Automatic categorization and validation

v3.0.1 - February 2027 (Remote Monitoring Integration)

Focus: Inverter APIs, real-time monitoring, proactive alerting

Features:

- **✔ Inverter API Integrations**
 - Growatt API for inverter data
 - Victron API for solar systems
 - Deye/Sunsynk API support
 - Generic MQTT integration
- **✔ Real-Time Monitoring**
 - Live inverter data dashboard
 - Real-time fault detection
 - Automatic alert generation
 - Historical data trend analysis
- **✔ Proactive Maintenance**
 - Fault prediction based on patterns
 - Maintenance recommendations
 - Parts failure forecasting
 - Service interval recommendations

v3.0.2 - March 2027 (Advanced Warranty & Financial)

Focus: Warranty intelligence, financial operations, payout optimization

Features:

- ☒ **Warranty Analytics**
 - Fault trend analysis by product, manufacturer, age
 - Warranty reserve calculations
 - Cost optimization for warranty operations
 - Supplier performance tracking
 - ☒ **Installer Payout Excellence**
 - Payout calculation engine (tiered, bonuses, deductions)
 - Zoho Books integration for accounting
 - Payout history and analytics
 - Bonus eligibility tracking
 - ☒ **Financial Dashboards**
 - Revenue dashboard (by product, technician, branch)
 - Cost analysis (warranty, service, overhead)
 - Margin tracking per installation type
 - Profitability by customer segment
-

Beyond v3.0 - Future Opportunities (Not Blocking)

These features are aligned with HANNA vision but not on critical path:

Possible Future Features (v3.1+)

Mobile Applications:

- Technician mobile app (native iOS/Android)
- Customer mobile app for monitoring
- Retailer mobile app for on-the-go orders

Advanced Features:

- Voice-based WhatsApp ordering
- Augmented reality (AR) for installation guidance
- IoT integration for device tracking
- Advanced forecasting (demand, inventory)

Operational Excellence:

- Multi-tenant white-label capability (if expanding business model)
- Advanced permission and access control
- Custom workflow builder

- Integration marketplace for third-party apps

Intelligence:

- Computer vision for photo validation
- Natural language processing for support tickets
- Blockchain for warranty certificates (if needed)
- Digital signatures for compliance

Roadmap Alignment with HANNA Core Scope

 Core Scope Coverage

Core Vision	Roadmap Coverage	Quarter
Installation Lifecycle Management	v2.1-2.2 Frontend completion, automation	Q1-Q2
Multi-Installation Type Support	ISR system for SSI, SLI, CFI, Hybrid	Q2
Technician Field Operations	Checklist, photos, offline sync	Q2
Warranty & Service Management	Complete lifecycle, claims, certificates	Q3
E-Commerce Platform	Shopping, WhatsApp shop, checkout	Q4
Multi-Portal Architecture	All 6 portals enhanced over releases	Q1-Q4
Real-Time Operations	WebSocket, monitoring, alerts	Q2-Q3
AI-Powered Automation	Document processing, predictive models	Q3 v3.0
Business Intelligence	Analytics, reporting, KPIs	Q2-Q3
Integration Ecosystem	Zoho, Paynow, email, inverters	Q3-Q4

 Quarterly Goals

Q1 2026 (v2.1): Stabilize operations, notification system, backend reliability

Q2 2026 (v2.2): Complete ISR implementation, monitoring, analytics

Q3 2026 (v2.3): Warranty excellence, Zoho integration, scalability

Q4 2026 (v2.4): E-commerce completion, customer experience, reporting

Q1 2027 (v3.0): AI operations, predictive capabilities, remote monitoring

Release Quality Standards

For Each Release (Monthly Subversion):

-  All tests passing (target: >80% code coverage)
-  Performance benchmarks met (page load <2s, API response <500ms)
-  Security audit completed (no new vulnerabilities)
-  Documentation updated
-  User guides provided for new features

- ☒ Zero critical bugs identified
- ☒ Backward compatibility maintained (unless breaking change documented)

For Each Major Release (Quarterly):

- ☒ All minor version quality standards
 - ☒ Migration guide provided (if needed)
 - ☒ User training materials prepared
 - ☒ Stakeholder communication sent
 - ☒ Post-release monitoring plan defined
-

Success Metrics

System Health:

- 99.9% uptime (except scheduled maintenance)
- <100ms API response time (95th percentile)
- <2s page load time (95th percentile)
- <5% task failure rate in Celery queue

User Adoption:

- Admin portal: 100% of staff trained by end of Q1
- Technician portal: 100% field completion rate with checklists and photos
- Client portal: 75%+ customer adoption by end of Q2

Business Metrics:

- Installation completion time: <14 days (average)
 - Photo compliance: 100% by Q2
 - Warranty claim processing: <5 days average
 - Customer satisfaction: NPS >50
-

Flexibility & Adaptation

This roadmap is **aligned but not limiting**. We may:

- ☒ Add features if customer need arises
- ☒ Shift timing based on priorities
- ☒ Combine or split releases based on progress
- ☒ Fast-track critical features (e.g., bugs, compliance)
- ☒ Defer lower-priority items if blocking work exists

Monthly review cycle ensures we stay responsive to market and operational needs.

Contributors

Core Team:

- Backend Development: Django, Celery, AI Integration
- Frontend Development: React, Next.js, TypeScript
- DevOps: Docker, Nginx, SSL automation
- QA: Testing, security audits, documentation

Special Thanks:

- Meta WhatsApp Business API team
- Google Gemini AI team
- Open-source community contributors

Support & Contact

Technical Support:

- Email: support@pfungwa.co.zw
- WhatsApp: +263 XXX XXX XXX
- Documentation: <https://docs.hanna.co.zw>

Bug Reports:

- GitHub Issues: <https://github.com/yourusername/hanna/issues>
- Email: bugs@pfungwa.co.zw

Feature Requests:

- GitHub Discussions: <https://github.com/yourusername/hanna/discussions>
- Email: features@pfungwa.co.zw

License

Copyright © 2026 Pfungwa Technologies. All rights reserved.

This software is proprietary and confidential. Unauthorized copying, distribution, or use is strictly prohibited.

Acknowledgments

Version 2.0.0 represents months of development, testing, and refinement. We thank our beta testers, early adopters, and partners who provided valuable feedback throughout the development cycle.

This release is dedicated to:

- Installation technicians working across Zimbabwe
- Solar energy entrepreneurs bringing power to communities
- Customers trusting us with their installation journeys

Ready to deploy HANNA v2.0.0? Follow the deployment guide above and join us in revolutionizing installation lifecycle management across Africa! 

Last Updated: February 1, 2026

Document Version: 2.0.0

Platform: HANNA Installation Lifecycle Operating System

- **Notification Templates:** All WhatsApp notification templates have been renamed from `hanna_*` prefix to `pfungwa_*` prefix
- **Affected Systems:**
 - All flow definitions across solar, Starlink, cleaning, site inspection, and loan applications
 - Email integration notification handlers
 - Customer data signals and event handlers
 - Admin flow processors
 - Test suites and validation scripts

Migration Required:

- Database: Update all `NotificationTemplate` records with new naming convention
- Run: `python manage.py load_notification_templates` to update template definitions
- Run: `python manage.py seed_notification_templates --force` to refresh database entries
- Verify: `python manage.py sync_meta_templates --dry-run` to check Meta API sync status

New Features

Notification System Enhancements

1. New Templates Added:

- `pfungwa_solar_alert_notification` - Solar system monitoring alerts for technical teams
- `pfungwa_sla_alert` - SLA violation notifications for warranty request tracking
- `pfungwa_new_custom_furniture_installation_request` - Custom furniture delivery/installation workflow
- `pfungwa_new_loan_application` - Financial services loan application tracking
- `pfungwa_new_warranty_claim_submitted` - Warranty claim submission notifications
- `pfungwa_warranty_claim_status_updated` - Customer warranty claim status updates

2. Template Consolidation:

- Centralized all notification templates in `flows/management/commands/load_notification_templates.py`
- Unified template definitions across seed, load, and flow definition modules
- Eliminated template fragmentation and duplication

3. Enhanced Coverage:

- Installation requests (Solar, Starlink, Cleaning, Custom Furniture)
- Order lifecycle (creation, updates, payment status, dispatch)
- Warranty management (registration, expiration, claims)
- Service requests and job card automation
- Financial operations (payouts, commissions, invoices)

- System monitoring and alerts

Backend Architecture

1. Notification Template Infrastructure:

- Dual field support for `message_body` and `body` fields (backward compatibility)
- Enhanced template parameter mapping for Meta WhatsApp API
- Improved template validation and error handling

2. Flow Definitions Updated:

- `starlink_installation_flow.py` - Updated notification integration
- `solar_installation_flow.py` - Enhanced admin notifications
- `solar_cleaning_flow.py` - Improved request tracking
- `site_inspection_flow.py` - Streamlined assessment notifications
- `simple_add_order_flow.py` - Placeholder order alerts
- `loan_application_flow.py` - Financial services workflow
- `lead_gen_flow.py` - Online order placement tracking
- `admin_update_order_status_flow.py` - Customer status notifications
- `admin_update_assessment_status_flow.py` - Assessment progress updates
- `admin_add_order_flow.py` - Admin-initiated order notifications

3. Signal & Task Updates:

- Updated `stats/signals.py` for human handover and order creation
- Enhanced `notifications/tasks.py` for 24-hour window reminders
- Improved `notifications/handlers.py` for message failure tracking
- Refined `email_integration/tasks.py` for invoice and job card processing
- Updated `customer_data/signals.py` for order event notifications

Frontend Improvements

1. Version Display:

- Added v2.0.0 version indicator in dashboard footer (React frontend)
- Added v2.0.0 version indicator in admin sidebar (Next.js management frontend)

2. Branding Consistency:

- Pfungwa Technologies branding maintained across both frontends
- Consistent styling and user experience

Bug Fixes

1. Notification Template Issues:

- Fixed missing `solar_alert_notification` template definition
- Resolved `sla_alert` template absence in warranty tracking
- Corrected template naming inconsistencies across flow definitions

2. Template Parameter Handling:

- Fixed empty parameter handling in notification rendering
- Improved Jinja2 template validation
- Enhanced error messaging for missing template variables

3. Flow Integration:

- Corrected template references in WhatsApp flow response processor
- Fixed notification queue context passing
- Improved related_contact association in group notifications

Technical Improvements

Database & ORM

- Enhanced NotificationTemplate model field compatibility
- Improved template context serialization for JSON fields
- Better handling of null/empty parameter values

Testing & Quality

- Updated test fixtures with new template naming convention
- Enhanced template rendering test coverage
- Improved validation for Meta API template format compliance
- Added docstring updates across test suites

Code Organization

- Centralized template definitions for easier maintenance
- Removed duplicate template lists from multiple files
- Improved separation of concerns between load, seed, and sync commands
- Better documentation in template definition files

Performance

- Optimized template lookup and validation
- Reduced duplicate database queries in notification processing
- Improved caching for frequently accessed templates

Statistics

- **Templates Renamed:** 40+ notification templates
- **Files Modified:** 26 files across backend and frontend
- **Lines Changed:** 2,000+ lines (template definitions and references)
- **Apps Affected:**
 - flows (8 definition files)
 - notifications (5 files)

- email_integration (1 file)
- customer_data (2 files)
- products_and_services (1 file)
- stats (1 file)
- warranty (1 file)
- solar_integration (1 file)

Deployment Notes

Pre-Deployment Checklist

- ☐ Backup production database (especially `notifications_notificationtemplate` table)
- ☐ Review all custom notification templates for naming conflicts
- ☐ Test notification delivery in staging environment
- ☐ Verify Meta WhatsApp API template sync status
- ☐ Check webhook configurations for notification triggers

Deployment Steps

1. Backend Deployment:

```
# Pull latest code
git pull origin main

# Activate virtual environment
cd whatsappcrm_backend
source venv/bin/activate # or venv\Scripts\activate on Windows

# Install dependencies (if any new)
pip install -r requirements.txt

# Run migrations (if any)
python manage.py migrate

# Load updated notification templates
python manage.py load_notification_templates

# Force update seed templates
python manage.py seed_notification_templates --force

# Sync with Meta (dry-run first)
python manage.py sync_meta_templates --dry-run
python manage.py sync_meta_templates # if dry-run successful

# Restart services
docker-compose restart backend celery_io_worker celery_cpu_worker
```

2. Frontend Deployment (Dashboard):

```
cd whatsapp-crm-frontend
npm install # if dependencies changed
npm run build
docker-compose restart frontend
```

3. Frontend Deployment (Management):

```
cd hanna-management-frontend
npm install # if dependencies changed
npm run build
# Deploy according to your Next.js hosting setup
```

Post-Deployment Verification

- ☐ Verify template names in admin panel match new convention
- ☐ Test sample notification delivery (human handover, order creation)
- ☐ Check Meta WhatsApp API template sync status
- ☐ Monitor error logs for any template-not-found errors
- ☐ Verify version numbers display correctly in both frontends
- ☐ Test one notification from each category (order, installation, warranty, etc.)

Rollback Plan

If issues occur:

1. Restore database from backup
2. Revert code to previous version: `git revert <commit-hash>`
3. Restart all services
4. Run previous version's `load_notification_templates`

Migration Guide

For Developers

If you have custom code referencing notification templates:

Before:

```
queue_notifications_to_users(  
  template_name='hanna_new_order_created',  
  # ...  
)
```

After:

```
queue_notifications_to_users(  
    template_name='pfungwa_new_order_created',  
    # ...  
)
```

For Database Admins

Update any direct database queries or scripts that filter by template name:

Before:

```
SELECT * FROM notifications_notificationtemplate  
WHERE name LIKE 'hanna_%';
```

After:

```
SELECT * FROM notifications_notificationtemplate  
WHERE name LIKE 'pfungwa_%';
```

Future Roadmap

Planned for v2.1.0

- Multi-language support for notification templates
- Template versioning and A/B testing framework
- Enhanced analytics for notification delivery rates
- SMS fallback for failed WhatsApp notifications
- Customer notification preference management

Under Consideration

- Email template integration with unified notification system
- Push notification support for mobile apps
- Template editor UI in admin dashboard
- Scheduled notification campaigns
- Advanced template personalization with ML

Acknowledgments

Special thanks to the development team for executing this major refactoring effort:

- Template consolidation and migration
- Comprehensive testing across all notification paths

- Documentation updates
 - Smooth coordination across multiple codebases
-

Support

For issues or questions related to this release:

- **Technical Issues:** Check the troubleshooting section in docs/troubleshooting/
 - **Template Questions:** Refer to FLOW_INTEGRATION_GUIDE.md
 - **General Support:** Contact development team
-

Additional Resources

- [Template Integration Guide](#)
 - [Notification System Documentation](#)
 - [Meta WhatsApp API Setup](#)
 - [Development Guide](#)
-

Full Changelog: <https://github.com/morebnyemba/hanna/compare/v1.0.0...v2.0.0>

Contributors: Pfungwa Technologies Development Team